8 Memoria Dinamica e Liste

Questa dispensa approfondisce la gestione dinamica della memoria in C e l'implementazione delle liste concatenate, strutture dati fondamentali quando la quantità di dati da gestire non è nota a priori.

8.1 Allocazione Dinamica della Memoria

In un programma C, la memoria è partizionata in diverse zone. Le variabili statiche e automatiche (quelle locali alle funzioni) risiedono nello **Stack**. Tuttavia, spesso non si conosce il numero esatto di dati da allocare fino al momento dell'esecuzione (run-time).

Per gestire questi casi, si utilizza lo **Heap** (Mucchio), un'area di memoria dedicata alle variabili dinamiche. A differenza delle variabili automatiche, quelle dinamiche:

- Vengono allocate e de-allocate esplicitamente dal programmatore.
- Sono accessibili esclusivamente tramite puntatori.

Funzioni `malloc` e `free` La libreria standard `stdlib.h` fornisce le funzioni necessarie per l'allocazione dinamica della memoria.

- `malloc` **(Memory Allocation)**: Alloca un blocco di byte contigui nello Heap. Restituisce un puntatore generico `void*` all'indirizzo base del blocco allocato. Se lo spazio non è sufficiente, restituisce `NULL`.
- `free`: De-alloca un blocco di memoria precedentemente riservato, rendendolo disponibile per nuovi utilizzi.

Esempio di allocazione di un intero:

```
int *ptr;
// Allocazione nello Heap: sizeof calcola i byte necessari
ptr = (int*) malloc(sizeof(int));

if (ptr != NULL) {
    *ptr = 10;       // Utilizzo della variabile dinamica
```

```
    printf("%d", *ptr);
    free(ptr);      // Rilascio della memoria
}
```

8.2 Liste Concatenate

Le liste sono strutture dati dinamiche che possono crescere e decrescere durante l'esecuzione. A differenza degli array, gli elementi di una lista non sono necessariamente contigui in memoria; sono sparpagliati nello Heap e collegati logicamente tramite puntatori.

Ogni elemento (o nodo) contiene due parti fondamentali:

1. **Dato**: l'informazione utile.
2. **Link**: un puntatore al nodo successivo.

L'ultimo nodo della lista ha il puntatore impostato a `NULL` per indicare la fine della sequenza.

8.2.1 Definizione del Tipo

La definizione della struttura è ricorsiva (o auto-referenziale), poiché contiene un puntatore a una struttura dello stesso tipo.

```
typedef struct EL {
    int dato;           // Informazione (es. intero)
    struct EL * prox;   // Link al prossimo elemento
} ElemLista;

typedef ElemLista * ListaDiElem; // Puntatore alla testa
```

8.3 Operazioni sulle Liste

Le operazioni vengono implementate tramite funzioni che manipolano i puntatori. Spesso queste funzioni restituiscono il nuovo puntatore alla testa della lista, poiché le operazioni di inserimento o cancellazione potrebbero modificarlo.

8.3.1 Inizializzazione e Calcolo Dimensione

Una lista vuota è rappresentata semplicemente dal valore `NULL`.

```
ListaDiElem inizializza() {
    return NULL; // Lista vuota
}

// Calcolo della dimensione (Versione Iterativa)
int dimensione(ListaDiElem lista) {
    int count = 0;
    while (lista != NULL) {   // Scorre fino alla fine
        lista = lista->prox;  // Avanza al prossimo nodo
        count++;
    }
    return count; // Restituisce il conteggio
}
```

Esiste anche una versione ricorsiva, basata sull'induzione: la lunghezza è 0 se la lista è vuota, altrimenti è 1 + la lunghezza del resto della lista.

```
int dimensione(ListaDiElem lista) {
    // 1. Caso base: siamo alla fine della lista (o vuota)
    if (lista == NULL) {
        return 0;
    } else {
        // 2. Passo ricorsivo: 1 (me stesso) + la dimensione del
           resto
        return 1 + dimensione(lista->prox);
    }
}
```

8.3.2 Inserimento

L'inserimento richiede l'allocazione dinamica di un nuovo nodo e l'aggiornamento dei collegamenti.

Inserimento in Testa È l'operazione più efficiente ($O(1)$). Il nuovo nodo punta alla vecchia testa, e diventa la nuova testa.

```
ListaDiElem inserisciTesta(ListaDiElem lista, int elem) {
    ListaDiElem punt;
    punt = (ListaDiElem) malloc(sizeof(ElemLista)); // Alloca
    if(punt == NULL) {
        // Gestione errore di allocazione
```

```
        return lista; // Ritorna la lista originale
    }

    punt->dato = elem;
    punt->prox = lista; // Collega al vecchio primo elemento

    return punt;        // Ritorna il nuovo primo elemento
}
```

Inserimento in Coda Richiede la scansione di tutta la lista per trovare l'ultimo elemento e agganciarvi il nuovo nodo.

```
ListaDiElem inserisciCoda(ListaDiElem lista, int elem) {
    ListaDiElem punt, cur = lista;
    punt = (ListaDiElem) malloc(sizeof(ElemLista));
    punt->dato = elem;
    punt->prox = NULL; // SarÃ  l'ultimo

    if (lista == NULL) {
        return punt; // Se vuota, il nuovo nodo Ã¨ la testa
    } else {
        while (cur->prox != NULL) // Cerca l'ultimo
            cur = cur->prox;
        cur->prox = punt; // Aggancia
        return lista;
    }
}
```

Inserimento Ordinato Mantiene la lista ordinata cercando la posizione corretta. Richiede due puntatori: `puntCor` (corrente) e `puntPrec` (precedente) per gestire l'inserimento nel mezzo.

```
ListaDiElem inserisciOrd(ListaDiElem lista, int elem) {
    ListaDiElem punt, puntCor = lista, puntPrec = NULL;

    // Trova la posizione
    while (puntCor != NULL && elem > puntCor->dato) {
        puntPrec = puntCor;
        puntCor = puntCor->prox;
    }
    punt = (ListaDiElem) malloc(sizeof(ElemLista));
    punt->dato = elem;
    if (puntPrec != NULL) { // Inserimento interno o in coda
        punt->prox = puntCor;
        puntPrec->prox = punt;
        return lista;
```

```
    } else { // Inserimento in testa
        punt->prox = lista;
        return punt;
    }
}
```

8.3.3 Cancellazione

Per rimuovere un nodo, bisogna "scavalcarlo" aggiornando il puntatore del nodo precedente, e poi liberare la memoria con `free`.

Ecco un esempio di funzione ricorsiva per cancellare la prima occorrenza di un elemento:

```
ListaDiElem cancella(ListaDiElem lista, int elem) {
    ListaDiElem puntTemp;
    if (lista != NULL) {
        if (lista->dato == elem) {
            // Elemento trovato: salva il prossimo e libera il
               corrente
            puntTemp = lista->prox;
            free(lista);
            return puntTemp; // Il prossimo diventa la testa (o il
                link corrente)
        } else {
            // Ricorsione sul resto della lista
            lista->prox = cancella(lista->prox, elem);
        }
    }
    return lista;
}
```

8.3.4 De-allocazione Completa

Quando una lista non serve più, è fondamentale liberare tutta la memoria allocata scorrendo la lista e chiamando `free` su ogni nodo.

```
void eliminaLista(ListaDiElem lista) {
    ListaDiElem temp;
    while (lista != NULL) {
        temp = lista->prox; // Salva il riferimento al prossimo
        free(lista);        // Libera il corrente
        lista = temp;       // Avanza
    }
}
```

8.4 Esercizi

Esercizio 8.1

Data una lista di interi, scrivere una funzione che cerca il minimo elemento pari e restituisce il puntatore all'area di memoria contenente il nodo associato a quel valore.

Esercizio 8.2

Si consideri una lista di interi. Scrivere una funzione che riceve come parametro la lista ed un intero M e che restituisce la somma dei soli valori della lista che sono multipli di M. Se la lista è vuota, la funzione deve restituire -1.

Esercizio 8.3

Scrivere una funzione che elimina gli elementi duplicati da una lista di interi. Si implementi sia la versione iterativa che quella ricorsiva.

Esercizio 8.4

Scrivere una funzione che, ricevuta in ingresso una lista di interi, verifichi se l'andamento della lista è monotono crescente (ogni elemento è strettamente maggiore al precedente).

Esercizio 8.5

Si consideri una lista di interi concatenata. Scrivere una funzione che inverte la lista. Due soluzioni sono possibili:

- la funzione alloca in memoria una nuova lista (invertita) e poi (se richiesto) elimina la lista originale;
- la funzione riusa i nodi della lista per crearne una nuova invertita.

Esercizio 8.6

Definire una funzione che, data una lista opportunamente definita ed un intero n, inserisce n dopo l'ultimo elemento della lista maggiore di n. Se la lista non contiene alcun elemento maggiore di n, la procedura lascia la lista inalterata.

Esercizio 8.7

Si consideri una lista di interi. Definiamo picchi della lista quei valori che sono preceduti e seguiti da valori più piccoli della loro metà. Il primo e l'ultimo elemento della lista non possono essere picchi. Scrivere una funzione che data una lista restituisce il numero di picchi della lista.

Esercizio 8.8

Si considerino le seguenti definizioni:

```
// Lista di interi
typedef struct el{
    int dato;
    struct el* prox;
} Node;
typedef Node* Lista;

// Lista compressa di interi
typedef struct c_el{
    int dato, count;
    struct c_el* prox;
} CNode;
typedef CNode* ListaCompressa;
```

Le liste compresse rappresentano le classiche liste con nodi in cui appare il valore e il numero di volte che appare nella lista originale.

Esempio: la lista `[3,3,3,5,5,2,2,2,2]` viene rappresentata come `[(3,3),(5,2),(2,4)]`. Scrivere una funzione che prende una lista e costruisce la sua rappresentazione compressa.

Esercizio 8.9

Si scriva una funzione che calcola la differenza simmetrica degli elementi di due liste ordinate in senso crescente e prive di duplicati, restituendola come una nuova lista (allocata allo scopo), anch'essa ordinata. La differenza simmetrica è costituita dagli elementi che appartengono a una delle due liste ma non all'altra lista (contiene cioè tutti gli elementi che non sono in comune alle due liste).

Esercizio 8.10

Scrivere una funzione che, ricevendo in ingresso una lista dinamica e un intero $k > 0$, individui il nodo che si trova k posizioni dopo il nodo centrale e ne restituisca il puntatore al chiamante. Se tale nodo non esiste, ad esempio a causa di un'insufficiente numero di nodi nella lista, la funzione deve restituire il valore NULL.

BONUS: scrivere una soluzione che prevede una sola scansione della lista (una scansione intera più una scansione anche solo parziale è considerata equivalente a due scansioni).

Esercizio 8.11

Si scriva una funzione che verifichi se l'andamento di una lista è *ondulatorio*, cioè se non capita mai che tre numeri consecutivi siano in ordine crescente o decrescente. La funzione restituisce 1 se l'andamento è ondulatorio, 0 altrimenti.

Esercizio 8.12

Si definisca una struttura dati per una lista concatenata in cui ogni nodo è costituito da un valore intero, un puntatore al nodo successivo e un puntatore opzionale (`child`) che identifica la testa di una sotto-lista gerarchica. Si scriva un programma che costruisce tale struttura e implementa una funzione di inserimento e di stampa che gestisca la gerarchizzazione della struttura ed una funzione di "appiattimento". (flatten) che linearizza la gerarchia: ogni sotto-lista puntata da `child` deve essere inserita nella lista principale immediatamente dopo il nodo genitore, modificando i puntatori esistenti senza allocare nuovi nodi. Il programma visualizzi a video la sequenza lineare risultante.

Esempio: data la lista gerarchica

```
1 -> 2 -> 5
     |
     3 -> 4
          |
          10
```

la lista appiattita sarà:

```
1 -> 2 -> 3 -> 4 -> 10 -> 5
```

Soluzioni

Soluzione dell'esercizio 8.1

```
#include "lista.c"

// Prototipo
ListaDiElem minimoPari(ListaDiElem);

// Funzione
ListaDiElem minimoPari(ListaDiElem head){
    if(head == NULL){
        printf("Lista vuota.\n");
        return NULL;
    }

    ListaDiElem curr = head, pMin = NULL;
    int min = 0;

    while(curr != NULL){
        if(curr->dato % 2 == 0){
            if(pMin == NULL || min > curr->dato){
                pMin = curr;
                min = curr->dato;
            }
        }
        curr = curr->prox;
    }

    return pMin;
}
```

Soluzione dell'esercizio 8.2

```
#include "lista.c"

int Somma(ListaDiElem lista, int M) {
        int sum = 0;
        if (lista == NULL)
                return -1;
        else {
                while(lista != NULL) {
                        if (lista->dato % M == 0)
                                sum = sum + lista->dato;
                        lista = lista->prox;
                }
                return sum;
        }
}

int Somma_rc(ListaDiElem lista, int M) {
        if (lista == NULL)
                return -1;
        if (lista->prox == NULL) {
                if(lista->dato % M == 0)
                        return lista->dato;
                return 0;
        }
```

```
        else {
                if(lista->dato % M == 0)
                        return lista->dato + Somma_rc(lista->prox, M);
                else
                        return Somma_rc(lista->prox, M);
        }
}
```

Soluzione dell'esercizio 8.3

```
#include "lista.c"

ListaDiElem eliminaDuplicatiIterativo(ListaDiElem head) {
    // Lista vuota o con un solo elemento
    if (head == NULL || head->prox == NULL) return head;

    ListaDiElem curr = head;

    // Scorro la lista esterna
    while (curr != NULL) {
        ListaDiElem runner = curr; // Runner per scorrere il resto della lista
        while (runner->prox != NULL) {
            if (runner->prox->dato == curr->dato) {
                // Nodo duplicato trovato
                ListaDiElem tmp = runner->prox;
                runner->prox = runner->prox->prox; // Salta il duplicato
                free(tmp); // Libera il nodo duplicato
            } else {
                runner = runner->prox; // Avanza
            }
        }
        curr = curr->prox; // Avanza nella lista esterna
    }

    return head;
}

ListaDiElem eliminaElementiUguali(ListaDiElem head, int value) {
    // Caso base: lista vuota
    if (head == NULL) return NULL;

    // Caso: il nodo corrente contiene 'value'
    if (head->dato == value) {
        ListaDiElem tmp = head;          // Salvo il nodo corrente
        head = eliminaElementiUguali(head->prox, value); // Elimino il resto dei
            nodi con 'value'
        free(tmp);                 // Libero il nodo corrente
        return head;
    }

    // Caso: il nodo corrente non contiene 'value'
    head->prox = eliminaElementiUguali(head->prox, value);
    return head;
}

ListaDiElem eliminaDuplicatiRicorsivo(ListaDiElem head) {
    // Caso base: lista vuota o con un solo elemento
    if (head == NULL || head->prox == NULL) return head;

    // Elimino ricorsivamente i duplicati del resto della lista
```

```
    head->prox = eliminaDuplicatiRicorsivo(head->prox);

    // Elimino ricorsivamente tutti i nodi uguali al dato di head
    head->prox = eliminaElementiUguali(head->prox, head->dato);

    return head;
}
```

Soluzione dell'esercizio 8.4

```
#include <stdio.h>
#include <stdlib.h>
#include "lista.c"

#define N 5


int monotona(ListaDiElem l);
int monotona_ric(ListaDiElem l);


int main () {
        ListaDiElem l=NULL;

        l=inserisciTesta(l,5);
        l=inserisciTesta(l,4);
        l=inserisciTesta(l,3);

        stampaLista(l);
        printf("Res: %d\n",monotona_ric(l));

        l=inserisciTesta(l,3);
        stampaLista(l);

        printf("Res: %d\n",monotona_ric(l));


        return 0;
}

int monotona(ListaDiElem l){
        ListaDiElem c1=l, c2;

        if(c1!=NULL)
                c2=c1->prox;

        while(c2!=NULL){
                if(c2->dato>=c1->dato)
                        return 0;
                c1=c2;
                c2=c2->prox;
        }
        return 1;
}

int monotona_ric(ListaDiElem l){
        if(l==NULL || l->prox==NULL){
                return 1;
        }
        if(l->dato >= l->prox->dato)
```

```
                return 0;
        else{
                return monotona_ric(l->prox);
        }
}
```

Soluzione dell'esercizio 8.5

```
#include <stdio.h>
#include <stdlib.h>
#include "lista.c"

#define N 10

ListaDiElem Reverse1(ListaDiElem lista, int keep);
ListaDiElem Reverse2(ListaDiElem lista);

int main () {

        ListaDiElem lis = NULL;
        ElemLista * max;
        int i;

        for (i = N; i > 0; i--)
                lis = InsInTesta(lis, i);

        stampaLista(lis);
        printf("\n");

        lis = Reverse1(lis, 0);

        stampaLista(lis);
        printf("\n");

        lis = Reverse2(lis);

        stampaLista(lis);
        printf("\n");

}

ListaDiElem Reverse1(ListaDiElem lista, int keep) {
   ListaDiElem temp = NULL, curr = lista;
   while(curr != NULL) {
      temp = inserisciTesta(temp, curr->dato);
      curr = curr->prox;
   }
   if(!keep)
      eliminaLista(lista);
   return temp;
}

ListaDiElem Reverse2(ListaDiElem lista) {
    ListaDiElem temp, prec = NULL;
    if(lista != NULL)  {
          while(lista->prox != NULL) {
             temp = prec;
             prec = lista;
             lista = lista->prox;
             prec->prox = temp;
```

```
          }
          lista->prox = prec;
    }
    return lista;
}

ListaDiElem Reverse2_rc(ListaDiElem lista) {
     ListaDiElem p, ris;
     if (lista == NULL || lista->prox == NULL)
          return lista;
     else {
          p = lista->prox;
          ris = Reverse2_rc(p);
          p->prox = lista;
          lista->prox = NULL;
          return ris;
     }
}
```

Soluzione dell'esercizio 8.6

```
#include<stdio.h>
#include<stdlib.h>
#include "lista.c"

#define N 5

ListaDiElem last_bigger(ListaDiElem, int);
void ins_last_bigger(ListaDiElem, int);

int main(){
        ListaDiElem l=NULL;
        int i;

        for(i=N;i>0;i--)
                l=inserisciTesta(l,i);
        stampaLista(l);

        ins_last_bigger(l,3);
        ins_last_bigger(l,12);
        ins_last_bigger(l,4);
        stampaLista(l);


        return 0;
}

ListaDiElem last_bigger(ListaDiElem l, int n){
        ListaDiElem cur=NULL;
        while(l!=NULL){
                if((l->dato)>n)
                        cur=l;
                l=l->prox;
        }
        return cur;
}


//NB: si può restituire void in questo caso perché la testa non può cambiare
void ins_last_bigger(ListaDiElem l, int n){
```

```
        ListaDiElem temp,new_el;

        temp=last_bigger(l, n);

        if(temp!=NULL){
                new_el=(ListaDiElem) malloc(sizeof(ElemLista));
                new_el->dato=n;
                new_el->prox=temp->prox;
                temp->prox=new_el;
        }
}
```

Soluzione dell'esercizio 8.7

```
#include <stdio.h>
#include <stdlib.h>
#include "lista.c"

#define N 10

typedef struct EL {
    float dato;
    struct EL * prox;
} ElemLista;

int Picchi(ListaDiElem lis);
int Picchi_rc(ListaDiElem lis);

int main () {

        ListaDiElem lis = NULL;

        lis = inserisciTesta(lis, 5);
        lis = inserisciTesta(lis, 11);
        lis = inserisciTesta(lis, 4);
        lis = inserisciTesta(lis, 9);
        lis = inserisciTesta(lis, 3);
        lis = inserisciTesta(lis, 5);

        stampaLista(lis);
        printf("\n");

        printf("Numero Picchi: %d\n", Picchi(lis));
        printf("Numero Picchi (Ricorsivo): %d\n", Picchi_rc(lis));

}

int Picchi(ListaDiElem lis) {
        int n = 0;
        while(lis != NULL && lis->prox != NULL && lis->prox->prox != NULL){

                if(((float)(lis->dato) < (float) (lis->prox->dato)/2)  &&
                        ((float)(lis->prox->prox->dato) < (float) (lis->prox->dato
                            )/2))
                        n++;

                lis = lis->prox;
        }
        return n;
}
```

```
int Picchi_rc(ListaDiElem lis) {
        if( !(lis != NULL && lis->prox != NULL && lis->prox->prox != NULL) )
        return 0;
    if(((float)(lis->dato) < (float) (lis->prox->dato)/2)  &&
                ((float)(lis->prox->prox->dato) < (float) (lis->prox->dato)/2))
                return 1 + Picchi_rc(lis->prox) ;
    else
        return Picchi(lis->prox) ;
}
```

Soluzione dell'esercizio 8.8

```
#include <stdio.h>
#include <stdlib.h>
#include "lista.c"


// Lista di interi
typedef struct el{
    int dato;
    struct el* prox;
} Node;
typedef Node* Lista;

// Lista compressa di interi
typedef struct c_el{
    int dato, count;
    struct c_el* prox;
} CNode;
typedef CNode* ListaCompressa;

ListaCompressa inserisciCodaCompresso(ListaCompressa head, int new_data, int
    new_count){
    // Alloco il nuovo nodo
    ListaCompressa new = (ListaCompressa)malloc(sizeof(CNode));
    if(new == NULL){
        printf("Errore di allocazione\n");
        return head;
    }
    new->dato = new_data;
    new->count = new_count;
    new->prox = NULL;

    // Verifico che la lista sia allcocata, altrimenti il nuovo nodo è il primo
    if(head == NULL) return new;

    // Scorro la lista fino alla fine
    ListaCompressa curr = head;
    while(curr->prox != NULL) curr = curr->prox;

    // curr è l'ultimo elemento, quindi attacco new
    curr->prox = new;

    return head;
}

int contaOccorrenze(Lista head, int value){
    int count = 0;
    Lista curr = head;
```

```
    while(curr != NULL){
        if(curr->dato == value) count++;
        curr = curr->prox;
    }
    return count;
}

int cercaElemento(ListaCompressa head, int value){
    int found = 0;
    ListaCompressa curr = head;
    while(curr != NULL && !found){
        if(curr->dato == value) found = 1;
        curr = curr->prox;
    }
    return found;
}

ListaCompressa comprimiLista(Lista head, ListaCompressa c_head){
    if(head == NULL) return c_head;

    Lista curr = head;
    int tmp_count;

    while(curr != NULL){
        if(!cercaElemento(c_head, curr->dato)){
            tmp_count = contaOccorrenze(curr, curr->dato);
            c_head = inserisciCodaCompresso(c_head, curr->dato, tmp_count);
        }
        curr = curr->prox;
    }

    return c_head;
}

void stampaListaCompressa(ListaCompressa head) {
    ListaCompressa curr = head;
    printf("Lista Compressa: [ ");
    while (curr != NULL) {
        printf("(%d, %d) ", curr->dato, curr->count);
        curr = curr->prox;
    }
    printf("]\n");
}

int main() {
    /* Dichiarazione variabili (Obbligatorio a inizio blocco in C89) */
    Lista myHead = NULL;
    ListaCompressa myCompHead = NULL;

    /* 1. Costruzione della lista dell'esempio: [3,3,3,5,5,2,2,2,2] */
    myHead = inserisciCoda(myHead, 3);
    myHead = inserisciCoda(myHead, 3);
    myHead = inserisciCoda(myHead, 3);
    myHead = inserisciCoda(myHead, 5);
    myHead = inserisciCoda(myHead, 5);
    myHead = inserisciCoda(myHead, 2);
    myHead = inserisciCoda(myHead, 2);
    myHead = inserisciCoda(myHead, 2);
    myHead = inserisciCoda(myHead, 2);

    /* 2. Stampa stato iniziale */
    stampaLista(myHead);
```

```
    /* 3. Chiamata alla funzione di compressione */
    /* Nota: passiamo NULL come secondo argomento perché la lista compressa parte
        vuota */
    myCompHead = comprimiLista(myHead, NULL);

    /* 4. Stampa risultato */
    stampaListaCompressa(myCompHead);

    /* 5. Pulizia memoria */
    eliminaLista(myHead);
    eliminaLista(myCompHead);

    return 0;
}
```

Soluzione dell'esercizio 8.9

```
#include<stdio.h>
#include<stdlib.h>
#include "lista.c"


ListaDiElem ds(ListaDiElem, ListaDiElem, ListaDiElem);
ListaDiElem ds_testa(ListaDiElem ,ListaDiElem);

int main(){
        ListaDiElem l1=NULL, l2=NULL, res=NULL;

        l1=inserisciCoda(l1,1);
        l1=inserisciCoda(l1,2);
        l1=inserisciCoda(l1,10);
        l1=inserisciCoda(l1,20);
        print_ListaDiElem(l1);

        l2=inserisciCoda(l2,2);
        l2=inserisciCoda(l2,3);
        l2=inserisciCoda(l2,4);
        l2=inserisciCoda(l2,10);
        print_ListaDiElem(l2);

        //res=ds(l1,l2,res);
        res=ds_testa(l1,l2);
        stampaLista(res);

        return 0;
}

ListaDiElem ds(ListaDiElem l1,ListaDiElem l2,ListaDiElem res){
        if(l1==NULL && l2==NULL)
                return res;
        if(l1==NULL || (l2!=NULL && l1->dato > l2->dato)){
                res = inserisciCoda(res,l2->dato);
                return ds(l1,l2->prox,res);
        }
        if(l2==NULL || (l1!=NULL && l2->dato > l1->dato)){
                res = inserisciCoda(res, l1->dato);
                return ds(l1->prox,l2,res);
        }
        return ds(l1->prox,l2->prox,res);
```

```
}

// variante con inserimento in testa
ListaDiElem ds_testa(ListaDiElem l1, ListaDiElem l2){
        ListaDiElem tmp;
        // entrambi finiti
        if(l1==NULL && l2==NULL)
                return NULL;
        // elementi uguali, scorro entrambe le liste
        if(l1!=NULL && l2!= NULL && l1->dato==l2->dato)
                return ds_testa(l1->prox, l2->prox);

        //se sono qui devo allocare un nuovo elemento
        tmp=(ListaDiElem)malloc(sizeof(ElemLista));

        if(l2==NULL || (l1!=NULL && l1->dato < l2->dato)){
                tmp->dato=l1->dato;
                l1=l1->prox;
        }else{
                tmp->dato=l2->dato;
                l2=l2->prox;
        }
        tmp->prox=ds_testa(l1,l2);
        return tmp;
}
```

Soluzione dell'esercizio 8.10

```
#include <stdio.h>
#include <stdlib.h>
#include "lista.c"



ListaDiElem trova_centro_k(ListaDiElem head, int k) {
    ListaDiElem fast = head;
    ListaDiElem target = head;
    int acc_vantaggio = 0; // Contatore per i passi di vantaggio accumulati

    if (head == NULL) return NULL;

    while (fast != NULL && fast->prox != NULL) {

        /* 1. Fast si muove sempre di 2 passi */
        fast = fast->prox->prox;

        /* 2. Gestione del puntatore Target */
        if (acc_vantaggio < k) {
            /* Finché non abbiamo accumulato k passi di vantaggio rispetto
               al "ritmo dimezzato", target corre alla stessa velocità di fast (2
                   passi). */
            target = target->prox->prox;
            acc_vantaggio++;
        } else {
            /* Abbiamo già il vantaggio k. Ora target si muove al ritmo
               standard del "middle finding" (1 passo alla volta). */
            target = target->prox;
        }
    }
```

```
    /* Se il ciclo è finito ma non abbiamo fatto in tempo ad accumulare
       tutto il vantaggio k (perché la lista era troppo corta),
       significa che il nodo (N/2 + k) non esiste. */
    if (acc_vantaggio < k) {
        return NULL;
    }

    return target;
}



int main(){
    ListaDiElem head = NULL; // Inizializzazione della lista
    ListaDiElem res = NULL;
    int err;

    printf("Creazione lista con inserimento in testa:\n");
    head = inserisciTesta(head, 1);
    head = inserisciTesta(head, 2);
    head = inserisciTesta(head, 3);
    head = inserisciTesta(head, 4);
    head = inserisciTesta(head, 5);
    head = inserisciTesta(head, 6);
    head = inserisciTesta(head, 7);

    stampaLista(head);

    res = trova_centro_k(head, 2);

    stampaLista(res);
}
```

Soluzione dell'esercizio 8.11

```
#include<stdio.h>
#include<stdlib.h>
#include "lista.c"


int ondulatorio(ListaDiElem l);

int main(){

        ListaDiElem l=NULL;

        l=inserisciTesta(l,2);
        l=inserisciTesta(l,1);
        l=inserisciTesta(l,2);
        l=inserisciTesta(l,3);
        l=inserisciTesta(l,4);

        printf("check 1, res: %d\n", ondulatorio(l));

        l=NULL;
        l=inserisciTesta(l,2);
        l=inserisciTesta(l,4);
        l=inserisciTesta(l,1);
        l=inserisciTesta(l,3);
```

```
        printf("check 2, res: %d\n", ondulatorio(l));

        return 0;
}

int ondulatorio(ListaDiElem l){
        ListaDiElem iter,p1,p2; //p1 e p2 punteranno agli elementi precedenti
        int c=0;

        while(iter!=NULL){
                c++;
                if(c==1){
                        p1=iter;
                }else if (c==2){
                        p2=iter;
                }
                else{
                        if((p1->dato > p2->dato && p2->dato > iter->dato) ||
                                (p1->dato < p2->dato && p2->dato < iter->dato)){
                                return 0;
                        }else{
                                p1=p2;
                                p2=iter;
                        }
                }
                iter=iter->prox;
        }
        return 1;
}
```

Soluzione dell'esercizio 8.12

```
#include <stdio.h>
#include <stdlib.h>

// Definizione nuova struttura
typedef struct ELG {
    int dato;
    struct ELG * prox;   // Puntatore al nodo successivo (orizzontale)
    struct ELG * figlio; // Puntatore alla testa della sotto-lista (verticale)
} ElemGerarchico;

typedef ElemGerarchico * ListaGerarchica;

ListaGerarchica creaNodo(int valore) {
    ListaGerarchica nuovo = (ListaGerarchica)malloc(sizeof(ElemGerarchico));
    if (nuovo == NULL) {
        printf("Errore di allocazione\n");
        exit(1);
    }
    nuovo->dato = valore;
    nuovo->prox = NULL;
    nuovo->figlio = NULL;
    return nuovo;
}

/* Inserimento in coda standard (Adattamento di inserisciCoda) */
ListaGerarchica append(ListaGerarchica head, int valore) {
    ListaGerarchica nuovo = creaNodo(valore);
```

```
    if (head == NULL) return nuovo;

    ListaGerarchica curr = head;
    while (curr->prox != NULL) {
        curr = curr->prox;
    }
    curr->prox = nuovo;
    return head;
}

/* Stampa semplice per verifica (Adattamento di stampaLista) */
void stampaListaLineare(ListaGerarchica head) {
    if (head == NULL) {
        printf("Lista vuota!\n");
    } else {
        ListaGerarchica curr = head;
        while (curr != NULL) {
            printf("%d -> ", curr->dato);
            curr = curr->prox;
        }
        printf("NULL\n");
    }
}

// Fuznioni di stampa
void stampaGerarchica_rec(ListaGerarchica head, int livello) {
    ListaGerarchica curr = head;
    int i;

    while (curr != NULL) {
        /* 1. Stampa l'indentazione in base al livello corrente */
        for (i = 0; i < livello; i++) {
            printf("       "); /* 7 spazi per ogni livello di profondità */
        }

        /* 2. Stampa il simbolo grafico e il dato */
        if (livello > 0) {
            printf("|__> "); /* Simbolo per i figli */
        } else {
            printf("[LO] "); /* Simbolo per la radice (Livello 0) */
        }

        printf("[%d]\n", curr->dato);

        /* 3. Se il nodo ha un figlio, chiamata ricorsiva aumentando il livello */
        if (curr->figlio != NULL) {
            stampaGerarchica_rec(curr->figlio, livello + 1);
        }

        /* 4. Passa al prossimo nodo orizzontale (stesso livello) */
        curr = curr->prox;
    }
}

void stampaGerarchica(ListaGerarchica head) {
    printf("\n--- Visualizzazione Struttura Gerarchica ---\n");
    if (head == NULL) {
        printf("Lista vuota.\n");
    } else {
        stampaGerarchica_rec(head, 0);
    }
    printf("-----------------------------------------\n");
```

```
}


void aggiungiFiglio(ListaGerarchica padre, int val_figlio) {
    if (padre == NULL) return;

    if (padre->figlio == NULL) {
        // Se non ha figli, creo la testa della lista figli
        padre->figlio = creaNodo(val_figlio);
    } else {
        // Se ha già figli, uso la funzione append sulla lista 'figlio'
        padre->figlio = append(padre->figlio, val_figlio);
    }
}

void flatten(ListaGerarchica head) {
    ListaGerarchica curr = head;

    while (curr != NULL) {
        // Se il nodo corrente ha una lista figlio
        if (curr->figlio != NULL) {

            // 1. Salviamo il riferimento al "vecchio" prossimo nodo
            ListaGerarchica temp_prox = curr->prox;

            // 2. Troviamo la CODA della lista del figlio
            // (Logica simile a inserisciCoda per trovare l'ultimo elemento)
            ListaGerarchica coda_figlio = curr->figlio;
            while (coda_figlio->prox != NULL) {
                coda_figlio = coda_figlio->prox;
            }

            // 3. Ricolleghiamo i puntatori

            // Il 'prox' del padre diventa l'inizio del figlio
            curr->prox = curr->figlio;

            // La fine del figlio si collega al vecchio 'prox' del padre
            coda_figlio->prox = temp_prox;

            // 4. Annulliamo il puntatore figlio poiché ora è integrato nella
                lista principale
            curr->figlio = NULL;
        }

        // Avanziamo al prossimo nodo.
        // NOTA: Se abbiamo appena fatto il flatten, curr->prox è ora il primo
        // nodo della ex-lista figlio. Questo permette di processare
            ricorsivamente
        // anche i nipoti senza bisogno di chiamate a funzione ricorsive.
        curr = curr->prox;
    }
}

int main() {
    /* Costruiamo l'esempio:
       [1] -> [2] -> [5] -> NULL
              |
             [3] -> [4] -> NULL
                    |
                   [10] -> NULL
    */
```

```
    // Livello 1 (Lista principale)
    ListaGerarchica lista = NULL;
    lista = append(lista, 1);
    lista = append(lista, 2);
    lista = append(lista, 5);

    // Livello 2 (Aggiungo figli al nodo [2])
    // lista->prox è il nodo [2]
    ListaGerarchica nodoDue = lista->prox;
    aggiungiFiglio(nodoDue, 3);
    aggiungiFiglio(nodoDue, 4);

    // Livello 3 (Aggiungo figlio al nodo [4])
    // nodoDue->figlio è [3], nodoDue->figlio->prox è [4]
    ListaGerarchica nodoQuattro = nodoDue->figlio->prox;
    aggiungiFiglio(nodoQuattro, 10);

    printf("Prima del Flatten\n");
    printf("Nota: stampa solo il livello principale, i figli sono nascosti.\n");
    stampaListaLineare(lista);
    stampaGerarchica(lista);

    printf("\nEsecuzione Flatten\n");
    flatten(lista);

    printf("\nDopo il Flatten\n");
    stampaListaLineare(lista);

    eliminaLista(lista);
    return 0;
}
```

Libreria Liste

```c
// Librerie
#include <stdio.h>
#include <stdlib.h>

// Dati
typedef struct EL {
    int dato;
    struct EL * prox;
} ElemLista;
typedef ElemLista * ListaDiElem;

// Prototipi
void stampaLista(ListaDiElem);
ListaDiElem inserisciCoda(ListaDiElem, int);
ListaDiElem inserisciTesta(ListaDiElem, int);
ListaDiElem inserisciPos(ListaDiElem, int, int);
ListaDiElem inserisciOrd(ListaDiElem, int);
ListaDiElem inserisciOrdinatoNoRip(ListaDiElem, int);
int dimensione(ListaDiElem);
float listaMedia(ListaDiElem, int*);
int cercaElemento(ListaDiElem, int);
void eliminaLista(ListaDiElem);
ListaDiElem eliminaPos(ListaDiElem, int);
ListaDiElem eliminaElementiValore(ListaDiElem, int);
ListaDiElem eliminaElementiValore_ric(ListaDiElem, int);

// Main
int main() {
    ListaDiElem head = NULL; // Inizializzazione della lista
    int err;

    printf("Creazione lista con inserimento in testa:\n");
    head = inserisciTesta(head, 10);
    head = inserisciTesta(head, 20);
    head = inserisciTesta(head, 30);
    stampaLista(head);

    printf("\nInserimento in coda:\n");
    head = inserisciCoda(head, 40);
    head = inserisciCoda(head, 50);
    stampaLista(head);

    printf("\nInserimento in posizione:\n");
    head = inserisciPos(head, 25, 2); // Inserisce 25 alla posizione 2
    stampaLista(head);

    printf("\nInserimento ordinato senza duplicati:\n");
    head = inserisciOrdinatoNoRip(head, 35); // Inserisce ordinato
    head = inserisciOrdinatoNoRip(head, 10); // Tentativo duplicato
    stampaLista(head);

    printf("\nCalcolo della lunghezza e media della lista:\n");
    int len = dimensione(head);
    float media = listaMedia(head, &err);
    if (!err) {
        printf("Lunghezza: %d, Media: %.2f\n", len, media);
    } else {
        printf("Errore nel calcolo della media (lista vuota).\n");
    }
```

```
    printf("\nRicerca di un elemento nella lista:\n");
    int val = 25;
    if (cercaElemento(head, val)) {
        printf("Elemento %d trovato nella lista.\n", val);
    } else {
        printf("Elemento %d non trovato.\n", val);
    }

    printf("\nEliminazione del nodo in posizione 3:\n");
    head = eliminaPos(head, 3); // Elimina il nodo alla posizione 3
    stampaLista(head);

    printf("\nEliminazione di tutti i nodi con valore 40:\n");
    head = eliminaElementiValore(head, 40); // Elimina tutti i nodi con valore 40
    stampaLista(head);

    printf("\nLiberazione della lista:\n");
    eliminaLista(head);
    head = NULL;
    stampaLista(head); // Verifica che la lista sia vuota

    return 0;
}

// Funzioni
void stampaLista(ListaDiElem head){
    if(head == NULL){
        printf("Lista vuota!\n");
    }
    else{
        ListaDiElem curr = head;
        // Stampo fino al penultimo
        while(curr->prox != NULL){
            printf("%d ->", curr->dato);
            curr = curr->prox;
        }
        // stampo l'ultimo
        printf("%d\n", curr->dato);
    }
}

ListaDiElem inserisciCoda(ListaDiElem head, int new_dato){
    // Alloco il nuovo nodo
    ListaDiElem new = (ListaDiElem)malloc(sizeof(ElemLista));
    if(new == NULL){
        printf("Errore di allocazione\n");
        return head;
    }
    new->dato = new_dato;
    new->prox = NULL;

    // Verifico che la lista sia allcocata, altrimenti il nuovo nodo è il primo
    if(head == NULL) return new;

    // Scorro la lista fino alla fine
    ListaDiElem curr = head;
    while(curr->prox != NULL) curr = curr->prox;

    // curr è l'ultimo elemento, quindi attacco new
    curr->prox = new;
```

```
    return head;
}

ListaDiElem inserisciTesta(ListaDiElem head, int new_dato){
    // Alloco il nuovo nodo
    ListaDiElem new = (ListaDiElem)malloc(sizeof(ElemLista));
    if(new == NULL){
        printf("Errore di allocazione\n");
        return head;
    }
    new->dato = new_dato;
    new->prox = NULL;

    // Bisogna che il nuovo nodo sia la testa e bisogna restituire la testa
    new->prox = head;

    return new;
}

ListaDiElem inserisciPos(ListaDiElem head, int new_dato, int index) {
    // Controllo se l'indice è valido
    if (index < 0) {
        printf("Indice %d non valido\n", index);
        return head;
    }

    // Caso speciale: lista vuota
    if (head == NULL) {
        if (index == 0) {
            return inserisciTesta(head, new_dato);
        } else {
            printf("Lista vuota, impossibile inserire in posizione %d.\n", index);
            return head;
        }
    }

    // Caso speciale: inserimento in testa
    if (index == 0) {
        return inserisciTesta(head, new_dato);
    }

    // Alloco il nuovo nodo
    ListaDiElem new = (ListaDiElem)malloc(sizeof(ElemLista));
    if (new == NULL) {
        printf("Errore di allocazione\n");
        return head;
    }
    new->dato = new_dato;
    new->prox = NULL;

    // Altri casi: Scorro la lista per trovare la posizione
    int pos = 0;
    ListaDiElem curr = head, prec = NULL;

    while (curr != NULL && pos < index) {
        prec = curr;
        curr = curr->prox;
        pos++;
    }

    // Verifica se l'indice è fuori dai limiti
    if (pos != index) {
```

```
        printf("Indice %d fuori dai limiti.\n", index);
        free(new);
        return head;
    }

    // Inserimento del nodo
    prec->prox = new;
    new->prox = curr;

    return head;
}

ListaDiElem inserisciOrd(ListaDiElem lista, int elem) {
    ListaDiElem punt, puntCor = lista, puntPrec = NULL;

    // Trova la posizione
    while (puntCor != NULL && elem > puntCor->dato) {
        puntPrec = puntCor;
        puntCor = puntCor->prox;
    }

    punt = (ListaDiElem) malloc(sizeof(ElemLista));
    punt->dato = elem;

    if (puntPrec != NULL) { // Inserimento interno o in coda
        punt->prox = puntCor;
        puntPrec->prox = punt;
        return lista;
    } else { // Inserimento in testa
        punt->prox = lista;
        return punt;
    }
}

ListaDiElem inserisciOrdinatoNoRip(ListaDiElem head, int new_dato) {
    // Alloco il nuovo nodo
    ListaDiElem new_ElemLista = (ListaDiElem)malloc(sizeof(ElemLista));
    if (new_ElemLista == NULL) {
        printf("Errore di allocazione\n");
        return head;
    }
    new_ElemLista->dato = new_dato;
    new_ElemLista->prox = NULL;

    // Controllo se il valore è già presente
    ListaDiElem curr = head, prec = NULL;

    while (curr != NULL && new_dato > curr->dato) {
        prec = curr;
        curr = curr->prox;
    }

    // Se il valore è già presente, non inserisco nulla
    if (curr != NULL && curr->dato == new_dato) {
        printf("Valore %d già presente, non inserito.\n", new_dato);
        free(new_ElemLista);
        return head;
    }

    // Inserimento in testa (prec == NULL)
    if (prec == NULL) {
        new_ElemLista->prox = curr;
```

```
        head = new_ElemLista;
    }
    // Inserimento tra prec e curr
    else {
        prec->prox = new_ElemLista;
        new_ElemLista->prox = curr;
    }

    return head;
}

int dimensione(ListaDiElem head){
    int len = 0;
    ListaDiElem curr = head;

    while(curr != NULL){
        curr = curr->prox;
        len ++;
    }

    return len;
}

int listaMax(ListaDiElem head, int* err){
    // lista vuota
    if(head == NULL){
        printf("Lista vuota\n");
        *err = 1;
        return 0;
    }

    // altrimenti
    *err = 0;
    int max = head->dato;
    ListaDiElem curr = head;

    while(curr != NULL){
        if(max < curr->dato) max = curr->dato;
        curr = curr->prox;
    }

    return max;
}

float listaMedia(ListaDiElem head, int* err){
    // lista vuota
    if(head == NULL){
        printf("Lista vuota\n");
        *err = 1;
        return 0;
    }

    // altrimenti
    *err = 0;
    float media = 0;
    int len = 0;
    ListaDiElem curr = head;

    while(curr != NULL){
        media += curr->dato;
        curr = curr->prox;
        len++;
```

```
    }

    return media/len;
}

int cercaElemento(ListaDiElem head, int value){
    ListaDiElem curr = head;

    while(curr != NULL){
        if(curr->dato == value) return 1;
        curr = curr->prox;
    }

    return 0;
}

void eliminaLista(ListaDiElem head){
    ListaDiElem curr = head;

    while(head != NULL){
        curr = head;
        head = head->prox;

        free(curr);
    }
}

ListaDiElem eliminaPos(ListaDiElem head, int index) {
    // Controllo se l'indice è valido
    if (index < 0) {
        printf("Indice %d non valido\n", index);
        return head;
    }

    // Lista vuota
    if (head == NULL) {
        printf("Lista vuota, impossibile eliminare.\n");
        return head;
    }

    // Eliminazione della testa
    if (index == 0) {
        ListaDiElem temp = head;
        head = head->prox;
        free(temp);
        return head;
    }

    // Altri casi: scorro la lista
    ListaDiElem curr = head, prec = NULL;
    int pos = 0;

    while (curr != NULL && pos < index) {
        prec = curr;
        curr = curr->prox;
        pos++;
    }

    // Controllo se l'indice è fuori dai limiti
    if (curr == NULL) {
        printf("Indice %d fuori dai limiti.\n", index);
        return head;
```

```
    }

    // Rimuovo il nodo corrente
    prec->prox = curr->prox;
    free(curr);

    return head;
}

ListaDiElem eliminaElementiValore(ListaDiElem head, int value){
    ListaDiElem curr = head, prec = NULL, tmp;

    // Elimino in testa
    while(curr != NULL && curr->dato == value){
        tmp = curr;
        curr = curr->prox;

        free(tmp);
    }
    // Aggiorno la testa
    head = curr;

    // Elimino non in testa
    while(curr != NULL){
        if(curr->dato == value){
            tmp = curr;
            prec->prox = curr->prox;
            curr = curr->prox;

            free(tmp);
        }
        else{
            prec = curr;
            curr = curr->prox;
        }
    }

    return head;
}


ListaDiElem eliminaElementiValore_ric(ListaDiElem lista, int elem) {
    ListaDiElem puntTemp;
    if (lista != NULL) {
        if (lista->dato == elem) {
            // Elemento trovato: salva il prossimo e libera il corrente
            puntTemp = lista->prox;
            free(lista);
            return puntTemp; // Il prossimo diventa la testa (o il link corrente)
        } else {
            // Ricorsione sul resto della lista
            lista->prox = cancella(lista->prox, elem);
        }
    }
    return lista;
}
```